🛠️ Fix Please – Electronic Repair Service & Management System

ระบบแจ้งซ่อมและบริหารจัดการงานซ่อมอุปกรณ์อิเล็กทรอนิกส์ (Computer, Laptop, Smartphone) ครบวงจร พัฒนาด้วยสถาปัตยกรรมแยกระหว่าง Frontend (React + Vite + Tailwind CSS v4) และ Backend (Bun + Express + TypeScript + Drizzle ORM + PostgreSQL) พร้อมระบบยืนยันตัวตนด้วย HttpOnly Cookie, Rate Limiting และการแบ่งสิทธิ์ผู้ใช้งาน (Role-Based Access Control)

📋 สารบัญ (Table of Contents)

1. **ภาพรวมของระบบและบทบาทผู้ใช้งาน** (System Overview & Roles)
2. **โครงสร้างโปรเจกต์** (Project Structure)
3. **ความต้องการของระบบ** (System Requirements)
4. **ขั้นตอนการติดตั้งและเริ่มใช้งาน** (Getting Started)
5. **การจัดการฐานข้อมูล** (Database & Drizzle ORM)
6. **ตารางสรุป API Endpoints** (API Documentation)
7. **สถาปัตยกรรมความปลอดภัยและการคำนวณรายรับ** (Security & Revenue Logic)

🌟 ภาพรวมของระบบและบทบาทผู้ใช้งาน

1. ลูกค้าทั่วไป (Public Customers)

- **หน้าหลัก (Home):** ค้นหาบริการเด่น ดูหมวดหมู่การซ่อม ข้อดีของศูนย์บริการ และคลิกเพื่อแจ้งซ่อมได้ทันที
- **หน้ารวมบริการ (Services):** เลือกดูบริการซ่อมทั้งหมด ค้นหาตามชื่อ กรองตามหมวดหมู่ (Computer, Laptop, Phone) และกดปุ่ม *"ส่งซ่อมบริการนี้"* เพื่อส่งต่อไปยังฟอร์มแจ้งซ่อม
- **หน้าติดต่อเรา / แบบฟอร์มแจ้งซ่อม (About Me & Contact Us):**
  - **กรณีเข้าโดยตรง:** กรอกแบบฟอร์มได้ทันที และเลือกบริการที่ต้องการซ่อมจาก Dropdown โดยมีตัวเลือก *"ไม่แน่ใจอาการ / ปรึกษาช่างก่อน (ประเมินภายหลัง)"* ซึ่งจะส่งค่า `service_id` เป็น `null`
  - **กรณีกดมาจากหน้า Services/Home:** ระบบจะเลือก (Pre-selected) บริการนั้นๆ ใน Dropdown ให้อัตโนมัติ
  - รองรับการกรอกข้อมูลลูกค้า (ชื่อ, เบอร์โทร, อีเมล, อาการเสีย) และแนบ URL รูปถ่ายความเสียหาย (สำหรับระดับ Starter Project)
- **ดูรูปภาพขยาย (Image Lightbox):** สามารถคลิกดูรูปภาพประกอบการซ่อมหรือรูปที่แนบมาแบบเต็มจอได้

2. ช่างซ่อม (Technician)

- เข้าสู่ระบบผ่านหน้า `/login`
- **สิทธิ์การเข้าถึง:** เข้าถึงได้เฉพาะ **"กล่องข้อความแจ้งซ่อม (Admin Inbox)"** (`/admin/inbox`) เท่านั้น (ระบบจะทำการ Redirect ป้องกันการเข้าหน้าอื่นอัตโนมัติ)
- **ระบบกรองงานสำหรับช่าง:**
  - กรองงานตามหมวดหมู่ที่ตนเองรับผิดชอบ (Computer, Laptop, Phone)
  - กรองเฉพาะงานที่ *"ยังไม่ระบุบริการ (ลูกค้าไม่แน่ใจอาการ / null)"* หรือกรองตามช่างผู้รับผิดชอบ
- **การจัดการงานซ่อม:**
  - เปิดอ่านรายละเอียดของ Ticket (เรียก `PATCH /api/inbox/:id/read` เพื่อเปลี่ยนเป็นสถานะ "อ่านแล้ว")
  - ดูรูปภาพสินค้าที่ลูกค้าแนบมาผ่าน Lightbox
  - **แก้ไขได้เฉพาะงานของตนเองหรืองานที่ยังไม่มีผู้รับผิดชอบ:** ช่างไม่สามารถแก้ไขงานที่มอบหมายให้ช่างคนอื่นแล้ว และไม่สามารถลบใบแจ้งซ่อมได้
  - **เลือก/ผูกบริการที่ซ่อมจริง (Service Assignment):** ช่างสามารถเลือกบริการจริงที่ดำเนินการซ่อม เพื่อผูกราคากับระบบ
  - **บันทึกสถานะงานซ่อม:** เปลี่ยนสถานะเป็น *รอซ่อม (Pending)* หรือ *ซ่อมเสร็จแล้ว (Completed)* พร้อมใส่บันทึกความคิดเห็นของช่าง
  - **Validation Guard & Price Snapshot:** ป้องกันไม่ให้เปลี่ยนสถานะเป็น "ซ่อมเสร็จแล้ว" หากยังไม่ได้เลือกผูกบริการจริง และเมื่อปิดงานจะบันทึก `final_price` และ `completed_at` ทันที

3. ผู้ดูแลระบบ (Admin)

- มีสิทธิ์เข้าถึงระบบหลังบ้านครบทุกฟังก์ชัน (Full Access):
  1. **Dashboard (แดชบอร์ด):** สรุปยอดรายรับทั้งหมด, รายสัปดาห์ (7 วัน), รายเดือน (30 วัน) จาก `final_price` และ `completed_at`, จำนวนงานที่ซ่อมเสร็จ, จำนวนงานรอซ่อม, จำนวนช่างในระบบ, กราฟสัดส่วนรายรับตามหมวดหมู่, รายการกิจกรรมงานซ่อมล่าสุด และสถิติผลงานช่างแต่ละคน
  2. **จัดการบริการ (Services Management):** เพิ่ม, แก้ไข, ลบ, ค้นหา, กรอง และสลับสถานะเปิด/ปิดบริการซ่อม
  3. **กล่องข้อความแจ้งซ่อม (Inbox):** ค้นหา, กรองงานตามช่าง, มอบหมายช่างผู้รับผิดชอบ, อัปเดตสถานะ, และลบข้อความแจ้งซ่อม
  4. **จัดการผู้ใช้งาน (User Management):** เพิ่ม, แก้ไข, ลบผู้ใช้งานในระบบ กำหนด Role เป็น Admin หรือ ช่าง (Technician) พร้อมระบบตรวจสอบ Username ซ้ำ

📁 โครงสร้างโปรเจกต์

```
Starter-Website-Intern/
├── backend/                        # ฝั่ง Backend (Bun + Express + Drizzle ORM)
│   ├── src/
│   │   ├── config/
│   │   │   ├── db.ts               # การเชื่อมต่อ PostgreSQL (postgres.js + drizzle)
│   │   │   └── env.ts              # ตรวจสอบ JWT_SECRET และตั้งค่า CORS Whitelist (CLIENT_URL)
│   │   ├── controllers/            # Controller จัดการ Logic แต่ละ Entity
│   │   │   ├── auth.controller.ts        # ล็อกอิน, ล็อกเอาต์, ตรวจสอบ session (/api/auth)
│   │   │   ├── categories.controller.ts  # จัดการหมวดหมู่บริการ (/api/categories)
│   │   │   ├── dashboard.controller.ts   # คำนวณรายได้และสถิติภาพรวม (/api/dashboard)
│   │   │   ├── inbox.controller.ts       # จัดการกล่องข้อความแจ้งซ่อม (/api/inbox)
│   │   │   ├── services.controller.ts    # จัดการรายการบริการซ่อม (/api/services)
│   │   │   └── users.controller.ts       # จัดการบัญชีผู้ใช้งาน Admin/ช่าง (/api/users)
│   │   ├── db/
│   │   │   ├── schema.ts           # โครงสร้างตาราง Database, Enums และ Indexes (Drizzle Schema)
│   │   │   └── seed.ts             # สคริปต์ Seeding ข้อมูลเริ่มต้น (อ่านค่าจาก .env)
│   │   ├── middlewares/            # Middleware ตรวจสอบสิทธิ์, Rate Limit และดักจับ Error
│   │   │   ├── auth.middleware.ts        # ตรวจสอบ JWT Token และเช็กสถานะ/Role ล่าสุดจาก DB
│   │   │   ├── error.middleware.ts       # Global Error Handler
│   │   │   ├── rateLimit.middleware.ts   # จำกัดความถี่การยิง API (/auth/login, POST /inbox)
│   │   │   ├── role.middleware.ts        # เช็กสิทธิ์ requireAdmin, requireStaff
│   │   │   └── validate.middleware.ts    # ตรวจสอบ Request Body ด้วย Zod Schema
│   │   ├── routes/                 # Express Routers
│   │   │   ├── index.ts                  # รวม Route หลัก
│   │   │   ├── auth.routes.ts
│   │   │   ├── categories.routes.ts
│   │   │   ├── dashboard.routes.ts
│   │   │   ├── inbox.routes.ts
│   │   │   ├── services.routes.ts
│   │   │   └── users.routes.ts
│   │   ├── types/                  # TypeScript Types / Interfaces
│   │   ├── validators/             # Zod Validation Schemas
│   │   └── index.ts                # จุดเริ่มต้นเซิร์ฟเวอร์ Express (Entry Point)
│   ├── .env                        # ไฟล์เก็บ Environment Variables (ห้าม commit)
│   ├── .env.example                # ไฟล์ตัวอย่าง Environment Variables
│   ├── drizzle.config.ts           # การตั้งค่า Drizzle Kit CLI
│   ├── package.json
│   └── tsconfig.json
│
├── frontend/                       # ฝั่ง Frontend (React + Vite + Tailwind CSS v4)
│   ├── src/
│   │   ├── api/
│   │   │   └── client.ts           # Fetch API Client พร้อมส่ง HttpOnly Cookie อัตโนมัติ
│   │   ├── components/             # Reusable UI Components
│   │   ├── context/                # React Context Providers (AuthContext, ServiceContext)
│   │   ├── pages/                  # หน้าเพจทั้งหมด (Home, Services, AboutMe, Login, Admin)
│   │   ├── types/
│   │   │   └── index.ts            # Type definitions (Category, Service, User, InboxMessage)
│   │   ├── App.tsx                 # Route Configuration & Title Manager
│   │   ├── index.css               # Tailwind CSS v4 Styles
│   │   └── main.tsx                # React DOM Mount Point
│   ├── index.html
│   ├── package.json
│   ├── tsconfig.json
│   └── vite.config.ts
│
└── README.md                       # คู่มือเอกสารระบบฉบับสมบูรณ์
```

⚙️ ความต้องการของระบบ (System Requirements)

- **Runtime:** Bun (แนะนำเวอร์ชัน 1.1.0 ขึ้นไป) หรือ Node.js v18+
- **Database:** PostgreSQL (เวอร์ชัน 14 ขึ้นไป หรือ Neon Serverless Postgres)
- **Browser:** รองรับ Google Chrome, Microsoft Edge, Safari, Firefox เวอร์ชันปัจจุบัน

🚀 ขั้นตอนการติดตั้งและเริ่มใช้งาน (Getting Started)

1. ติดตั้ง Dependencies

เปิด Terminal ในโฟลเดอร์หลักของโปรเจกต์:

```
# ติดตั้ง dependencies ฝั่ง Backend
cd backend
bun install

# ติดตั้ง dependencies ฝั่ง Frontend
cd ../frontend
bun install
```

2. ตั้งค่าไฟล์ Environment Variables (`.env`)

ในโฟลเดอร์ `backend/` ให้สร้างไฟล์ `.env` โดยคัดลอกโครงสร้างมาจาก `.env.example` :

```
# Server Configuration
PORT=5000
NODE_ENV=development
# รายชื่อ Origin ที่อนุญาตให้เชื่อมต่อ CORS (คั่นด้วยลูกน้ำ , หากมีหลายโดเมน)
CLIENT_URL=http://localhost:3000

# PostgreSQL Connection String
DATABASE_URL=postgresql://<user>:<password>@localhost:5432/repair_service_db

# JWT & Cookie Secret Key (จำเป็นต้องระบุ หากไม่ระบุเซิร์ฟเวอร์จะไม่ยอมสตาร์ท)
JWT_SECRET=<your-strong-random-jwt-secret>

# ข้อมูลบัญชีเริ่มต้นสำหรับคำสั่ง Seed (bun run db:seed)
SEED_ADMIN_USERNAME=Admin
SEED_ADMIN_PASSWORD=<your-admin-password>
SEED_TECH_USERNAME=TechSomchai
SEED_TECH_PASSWORD=<your-technician-password>
```

3. เตรียมฐานข้อมูลและสร้างข้อมูลเริ่มต้น (Database Migration & Seed)

ซิงค์โครงสร้างตารางและรันคำสั่ง Seed ข้อมูลเริ่มต้น:

```
cd backend
bun run db:push
bun run db:seed
```

(ระบบจะสร้างหมวดหมู่เริ่มต้น, รายการบริการตัวอย่าง และบัญชีผู้ใช้งานตามที่ตั้งค่าไว้ใน `SEED*ADMIN*_` และ `SEED*TECH*_` ของไฟล์ `.env`)

4. รันระบบสำหรับการพัฒนา (Development Mode)

⚡ **วิธีที่ 1: รันคำสั่งเดียวพร้อมกันทั้งระบบ (One-liner Quick Start – แนะนำ)**

เปิด Terminal ที่โฟลเดอร์หลัก (Root) แล้วพิมพ์:

```
bun run start:all
```

🛠️ **วิธีที่ 2: รันแยกทีละหน้าต่าง Terminal**

Terminal 1 (Backend – พอร์ต 5000):

```
cd backend
bun run dev
```

Terminal 2 (Frontend – พอร์ต 3000):

```
cd frontend
bun run dev
```

เปิดเบราว์เซอร์ไปที่: `http://localhost:3000`

🗄️ การจัดการฐานข้อมูล (Database & Drizzle ORM)

ระบบใช้ **Drizzle ORM** ร่วมกับไดรเวอร์ `postgres.js` โดยมีคำสั่งจัดการฐานข้อมูลใน `backend/package.json` ดังนี้:

คำสั่ง	คำอธิบาย
`bun run db:push`	ซิงค์ Drizzle Schema ไปยัง PostgreSQL ทันที
`bun run db:generate`	สร้างไฟล์ SQL Migration ในโฟลเดอร์ `drizzle/`
`bun run db:migrate`	ดำเนินการรัน Migration ไปยังฐานข้อมูล
`bun run db:seed`	รันสคริปต์เพิ่มหมวดหมู่, บริการ และผู้ใช้เริ่มต้นจาก `.env`
`bun run db:studio`	เปิด Drizzle Studio (GUI ตรวจสอบข้อมูลในเบราว์เซอร์)

โครงสร้างตาราง (Database Schema Summary — ตรงตาม `backend/src/db/schema.ts`):

- **PostgreSQL Enums:**
  - `user_role`: `'admin'` | `'technician'`
  - `repair_status`: `'pending'` | `'completed'`

1. `categories` **(หมวดหมู่บริการ):**
   - `cg_id` (PK, Serial)
   - `cg_name` (Varchar 50, Not Null)
2. `services` **(รายการบริการซ่อม):**
   - `id` (PK, Serial)
   - `cg_id` (FK -> `categories.cg_id`, `onDelete: 'restrict'`, Not Null)
   - `name` (Varchar 100, Not Null)
   - `detail` (Varchar 500, Not Null)
   - `price` (Integer, Not Null)
   - `status` (Boolean, Default `true`, Not Null)
   - `service_image` (Varchar 255, Not Null)
   - **Indexes:** `idx_services_cg_id`, `idx_services_status`
3. `users` **(ผู้ใช้งานระบบหลังบ้าน):**
   - `u_id` (PK, Serial)
   - `username` (Varchar 50, Unique, Not Null)
   - `pwd` (Varchar 255, Bcrypt Hash, Not Null)
   - `u_name` (Varchar 50, Not Null), `u_surname` (Varchar 50, Not Null)
   - `role` (Enum `user_role`: `'admin'` | `'technician'`, Default `'technician'`, Not Null)
   - `cel` (Varchar 15, Not Null), `email` (Varchar 100, Not Null)
   - **Indexes:** `idx_users_role`
4. `inbox_messages` **(กล่องข้อความและรายการแจ้งซ่อม):**
   - `cm_id` (PK, Serial)
   - `service_id` (FK -> `services.id`, `onDelete: 'set null'`, Nullable)
   - `technician_id` (FK -> `users.u_id`, `onDelete: 'set null'`, Nullable)
   - `cm_username` (Varchar 100, Not Null), `cm_cel` (Varchar 15, Not Null), `cm_email` (Varchar 100, Not Null)
   - `cm_symptom` (Varchar 500, Not Null), `cm_image` (Varchar 255, Nullable)
   - `is_read` (Boolean, Default `false`, Not Null)
   - `repair_status` (Enum `repair_status`: `'pending'` | `'completed'`, Default `'pending'`, Not Null)
   - `final_price` (Integer, Nullable — บันทึกราคาบริการจริง ณ ตอนที่ปิดงานซ่อมเสร็จ)
   - `completed_at` (Timestamp, Nullable — บันทึกวันเวลาที่ซ่อมเสร็จจริง)
   - `comment` (Varchar 500, Nullable)
   - `created_at` (Timestamp, Default Now, Not Null)
   - **Indexes:** `idx_inbox_service_id`, `idx_inbox_technician_id`, `idx_inbox_repair_status`, `idx_inbox_is_read`, `idx_inbox_created_at`

📡 ตารางสรุป API Endpoints

Base URL: `http://localhost:5000/api` (Frontend เรียกผ่าน `/api`)

1. Authentication (`/api/auth`)

Method	Endpoint	สิทธิ์เข้าถึง	คำอธิบาย	Request Body
`POST`	`/api/auth/login`	Public (Rate Limited)	เข้าสู่ระบบ และเซ็ต HttpOnly Cookie (`auth_token`)	`{ "username": "...", "pwd": "..." }`
`POST`	`/api/auth/logout`	Public	เคลียร์ HttpOnly Cookie ออกจากเบราว์เซอร์	-
`GET`	`/api/auth/me`	Authenticated	ดึงข้อมูลผู้ใช้งานปัจจุบัน (ตรวจสอบกับฐานข้อมูลจริงทุกครั้ง)	-

2. หมวดหมู่บริการ (`/api/categories`)

Method	Endpoint	สิทธิ์เข้าถึง	คำอธิบาย	Request Body
`GET`	`/api/categories`	Public	ดึงรายการหมวดหมู่ทั้งหมด	-
`GET`	`/api/categories/:id`	Public	ดึงรายละเอียดหมวดหมู่ตาม ID	-
`POST`	`/api/categories`	Admin Only	เพิ่มหมวดหมู่ใหม่	`{ "cg_name": "Tablet" }`

3. บริการซ่อม (`/api/services`)

Method	Endpoint	สิทธิ์เข้าถึง	คำอธิบาย	Query / Request Body
`GET`	`/api/services`	Public	ดึงรายการบริการทั้งหมด	Query: `?cg_id=1&status=true`
`GET`	`/api/services/:id`	Public	ดึงรายละเอียดบริการตาม ID	-
`POST`	`/api/services`	Admin Only	เพิ่มบริการซ่อมใหม่	`{ "name": "...", "cg_id": 1, "price": 1500, "status": true, ... }`
`PUT`	`/api/services/:id`	Admin Only	แก้ไขข้อมูลบริการซ่อม	`{ "name": "...", "price": 1800, ... }`
`DELETE`	`/api/services/:id`	Admin Only	ลบบริการซ่อม	-

4. กล่องข้อความแจ้งซ่อม (`/api/inbox`)

Method	Endpoint	สิทธิ์เข้าถึง	คำอธิบาย	Query / Request Body
`POST`	`/api/inbox`	Public (Rate Limited)	ลูกค้าส่งแบบฟอร์มแจ้งซ่อม (`service_id` เป็น `null` ได้)	`{ "cm_username": "...", "cm_cel": "...", "cm_symptom": "...", "service_id"`
`GET`	`/api/inbox`	Staff (Admin/ช่าง)	ดึงรายการแจ้งซ่อมทั้งหมด พร้อม Left Join ชื่อบริการ, หมวดหมู่ และช่าง	Query: `?read_status=unread&category_id=1&service_id=unassigned&technician_id=...&se`
`GET`	`/api/inbox/:id`	Staff (Admin/ช่าง)	ดูรายละเอียด Ticket รายตัว (ไม่มี Side Effect)	-
`PATCH`	`/api/inbox/:id/read`	Staff (Admin/ช่าง)	อัปเดตสถานะข้อความเป็น "อ่านแล้ว (`is_read = true`)"	-
`PATCH`	`/api/inbox/:id/status`	Staff (Admin/ช่าง)	อัปเดตสถานะงาน (`repair_status`), ผูกบริการจริง (`service_id`), บันทึก `final_price` และ `completed_at` *(ช่างแก้ได้เฉพาะงานตัวเองหรืองานที่ยังไม่มีช่างรับ)*	`{ "repair_status": "completed", "service_id": 2, "comment": "..." }`
`DELETE`	`/api/inbox/:id`	Admin Only	ลบข้อความแจ้งซ่อมออกจากระบบ (เฉพาะ Admin)	-

5. ผู้ใช้งานระบบ (`/api/users`)

Method	Endpoint	สิทธิ์เข้าถึง	คำอธิบาย	Request Body
`GET`	`/api/users`	Staff (Admin/ช่าง)	ดึงรายชื่อผู้ใช้ทั้งหมด (ตัดรหัสผ่านออก) เพื่อใช้แสดง/กรองรายชื่อช่าง	-
`GET`	`/api/users/:id`	Admin Only	ดึงข้อมูลผู้ใช้งานตาม ID	-
`POST`	`/api/users`	Admin Only	สร้างบัญชีผู้ใช้ใหม่ (เข้ารหัสผ่านด้วย bcrypt)	`{ "username": "...", "pwd": "...", "role": "technician", ... }`
`PUT`	`/api/users/:id`	Admin Only	แก้ไขข้อมูลผู้ใช้ (เปลี่ยนรหัสผ่านได้เมื่อระบุ)	`{ "u_name": "...", "role": "admin" }`
`DELETE`	`/api/users/:id`	Admin Only	ลบบัญชีผู้ใช้งาน (มีระบบป้องกันการลบบัญชีตัวเอง)	-

6. แดชบอร์ดสรุปผล (`/api/dashboard`)

Method	Endpoint	สิทธิ์เข้าถึง	คำอธิบาย
`GET`	`/api/dashboard`	Admin Only	คำนวณสรุปยอดรายได้ (รวม/7วัน/30วัน) จาก `final_price` และ `completed_at`, จำนวนงานซ่อมเสร็จ, จำนวนช่าง, จำนวนงานรอซ่อม, สัดส่วนรายได้แยกตามหมวดหมู่, กิจกรรมล่าสุด และสถิติผลงานช่าง

🔒 สถาปัตยกรรมความปลอดภัยและการคำนวณรายรับ

1. **Strict JWT Secret & Real-time Database Session Verification:**
   - เซิร์ฟเวอร์บังคับให้ต้องตั้งค่า `JWT_SECRET` ใน `.env` เสมอ (ไม่มีรหัสสำรองฝังในโค้ด และจะไม่ยอมสตาร์ทหากไม่มีค่า)
   - Middleware `authenticate` ตรวจสอบ JWT จาก HttpOnly Cookie (`auth_token`) และดึงข้อมูลผู้ใช้พร้อม `role` ล่าสุดจากฐานข้อมูลทุกครั้ง หากผู้ใช้ถูกลบหรือถูกเปลี่ยน Role สิทธิ์จะมีผลทันทีโดยไม่ต้องรอ Token หมดอายุ
2. **CORS Origin Whitelist & Rate Limiting:**
   - ตั้งค่า CORS ให้รับเฉพาะ Origin ที่ระบุไว้ใน `CLIENT_URL` เท่านั้น
   - มี Rate Limiter ป้องกันการสุ่มรหัสผ่านที่ `POST /api/auth/login` และป้องกันการยิงสแปมฟอร์มแจ้งซ่อมที่ `POST /api/inbox`
3. **Role-Based Access Control (RBAC) & Ticket Ownership:**
   - ลบรายการแจ้งซ่อม (`DELETE /api/inbox/:id`) จำกัดสิทธิ์เฉพาะ `requireAdmin` เท่านั้น
   - ช่าง (`technician`) สามารถอัปเดตสถานะได้เฉพาะงานซ่อมที่ตนเองรับผิดชอบ หรือรายการที่ยังไม่มีช่างรับผิดชอบเท่านั้น
4. **การคำนวณรายรับใน Dashboard (Historical Revenue Integrity):**
   - เมื่อเปลี่ยนสถานะใบแจ้งซ่อมเป็น `'completed'` ระบบจะบันทึกราคาบริการ ณ ขณะนั้นลงในคอลัมน์ `final_price` และบันทึกเวลาปิดงานลงใน `completed_at`
   - หน้า Dashboard คำนวณยอดรายรับจาก `final_price` และกรองช่วงเวลา 7 วัน / 30 วันจาก `completed_at` ทำให้แม้ในอนาคตแอดมินจะแก้ไขราคาบริการหรือลบบริการทิ้ง ยอดรายรับย้อนหลังก็จะยังคงถูกต้อง 100% เสมอ